Consteval-only Values and Consteval Variables

Document #:	P3603R1 [Latest] [Status]
Date:	2025-10-06
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Peter Dimov <pdimov@gmail.com>

Contents

§ 1 Abstract

This paper formalizes the concept of *consteval-only value* and uses it to introduce consteval variables — variables that can only exist at compile time and are never code-gen. Consteval variables can then be used to solve some concrete problems we have today — like variant visitation.

§ 2 Revision History

Since [P3603R0]: added implementation experience and `not_fn` example, extended prose, updated wording now that reflection has been adopted.

§ 3 Introduction

C++ today already has an *informal* notion of a value that is only allowed to exist during compile time. Currently, this is ill-formed:

```
consteval int add(int x, int y) {
    return x + y;
}

constexpr auto ptr = add; // error
```

We cannot "persist" a pointer to immediate function like this — `add` is a value that is only allowed to exist at compile time. This is because we cannot invoke `ptr` at runtime, and if we allowed this, `ptr` would just be a regular old `int(*)(int, int)`. The original addition of `consteval` functions — [P1073R3] (Immediate functions) — already had this rule.

It's not just that you cannot create a `constexpr` variable whose value is `add`, you also cannot use it as a template argument, cannot have it as a member of a struct that's used in either way, etc.

Similarly, we cannot *really* persist reflections [P2996R13] (Reflection for C++26):

```
constexpr auto refl = ^^int;
```

We cannot allow you to do anything with `refl` at runtime in the same way we cannot do anything with `ptr` at runtime. But we enforce these requirements very differently:

— we disallow `ptr` through what used to be the "permitted result" rule

— we allow `refl` but ensure that all expressions involving `refl` are constant, by way of consteval-only types and immediate escalation from [P2564R3] (consteval needs to propagate up).

The status quo from the Reflection design is that we can handle these differently because we can differentiate based on type. `ptr` is just a function pointer, `refl` is a `std::meta::info` — we can ensure that expressions involving the latter are constant, but we can't tell from a given function pointer whether we need that machinery or not.

What if we did things a little bit differently?

§ 3.1 Consteval Variables

We currently have `consteval` functions (which can only exist at compile time) but we do not have `consteval` variables. What if we did?

We would have to say that a `consteval` variable could only exist at compile time, which means all uses of it must be constant. We already have these kinds of rules in the language, so it is straightforward to extend them to cover this case as well. That is, we would expect:

```
consteval int add(int x, int y) {
    return x + y;
}

// error: as before, cannot persist a pointer to immediate function
constexpr auto p1 = add;

// OK, p2 is a consteval variable. it does not exist at runtime
consteval auto p2 = add;

int main(int argc, char**) {
    // error: p2 is a consteval variable, so expressions using it must
        be
    // constant — and this is not.
    return p2(argc, 1);
}
```

This is already pretty nice. We could never initialize something like `p2` today (including as part of a struct, etc.), and this would allow us to.

Another important benefit of `consteval` variables is that they are *guaranteed* to not occupy space at runtime. You just don't hit issues like this. `constexpr` variables, even if never accessed at runtime, may occupy space anyway. It's just QoI. But in the same way that `consteval` functions *cannot* lead to codegen, `consteval` variables *cannot* either. That's a pretty nice benefit.

But how do we distinguish between what is allowed for `p1` and what is allowed for `p2`? We simply need to introduce…

§ 3.2 Consteval-Only Values

As mentioned earlier, we already have an implicit notion of consteval-only value in the language with how we treat immediate functions today. Let's make that more explicit, and also account for the consteval variables we're introducing. This isn't quite Core-precise wording, but it should convey the idea we need:

> An expression has a *consteval-only value* if:
>
> — it is a reflection,
>
> — it is a consteval variable,
>
> — it is an immediate function, or
>
> — any constituent part of it either points to or refers to a consteval-only value.

For instance, an *id-expression* naming an immediate function is a consteval-only value (like `add`), `^^int` is a consteval-only value, `members_of(^^something)` is a consteval-only value, `p2` in the above example is a consteval-only value (doubly so — both because it is a `consteval` variable and because it is a pointer to immediate function), etc.

Our rules around immediate-escalating expressions already presuppose the existence of a consteval-only value, this term just allows us to be more explicit about it:

> 25 An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and ~~it is~~ either
>
> (25.1) — ~~a potentially-evaluated *id-expression* that denotes an immediate function that~~ it has consteval-only value and it is not a subexpression of an immediate invocation, or
>
> (25.2) — it is an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.

This isn't just a way to clean up the specification a little. It also has some other interesting potential…

§ 3.3 Interaction with Other Papers

In [P3496R0] (Immediate-Escalating Expressions), we try to express that certain sub-expressions have to escalate. It achieves this by *also* introducing the notion of a consteval-only value — and saying that expressions that contain a consteval-only value have to escalate. While the goal of that paper is to have an expression (rather than a function) stop escalation, it needs to talk about this problem in the same way — so the addition of this terminology is clearly inherently useful for language evolution.

§ 4 Motivating Example: Variant Visitation

Jiang An submitted a very interesting bug report to libstdc++ (and libc++) in January 2025, which is also now [LWG4197]. It dealt with visiting a `std::variant` with a consteval lambda.

Here is a short reproduction of it, with a greatly reduced `variant` implementation that gets us to the point:

```
#include <array>

template <class T, class U>
struct Variant {
    union {
        T t;
        U u;
    };
    int index;

    constexpr Variant(T t) : t(t), index(0) { }
    constexpr Variant(U u) : u(u), index(1) { }
```

```
    template <int I> requires (I < 2)
    constexpr auto get() const -> auto const& {
        if constexpr (I == 0) return t;
        else if constexpr (I == 1) return u;
    }
};

template <class R, class F, class V0, class V1>
struct binary_vtable_impl {
    template <int I, int J>
    static constexpr auto visit(F&& f, V0 const& v0, V1 const& v1) -> R
        {
        return f(v0.template get<I>(), v1.template get<J>());
    }

    static constexpr auto get_array() {
        return std::array{
            std::array{
                &visit<0, 0>,
                &visit<0, 1>
            },
            std::array{
                &visit<1, 0>,
                &visit<1, 1>
            }
        };
    }

    static constexpr std::array fptrs = get_array();
};

template <class R, class F, class V0, class V1>
constexpr auto visit(F&& f, V0 const& v0, V1 const& v1) -> R {
    using Impl = binary_vtable_impl<R, F, V0, V1>;
    return Impl::fptrs[v0.index][v1.index]((F&&)f, v0, v1);
}

consteval auto func(const Variant<int, long>& v1, const Variant<int,
        long>& v2) {
     return visit<int>([](auto x, auto y) consteval { return x + y; },
        v1, v2);
}

static_assert(func(Variant<int, long>{42}, Variant<int, long>{1729}) ==
        1771);
```

Here, the lambda `[](auto x, auto y) consteval { return x + y; }` is `consteval`. It is invoked in multiple instantiations of `binary_vtable_impl<...>::visit<...>`, which causes those `constexpr` functions to escalate into `consteval` functions, due to [P2564R3] (otherwise the invocation would already be ill-formed). `get_array()` is returning a two-dimensional array of 4 function pointers into different instantiations of those functions, which are all `consteval` — and that array is stored as the `static constexpr` data member `fptrs`.

That is ill-formed.

Initialization of a `constexpr` variable (like `binary_vtable_impl<...>::fptrs` in this case) must be a constant expression, which must satisfy (from 7.7 [expr.const]/22, and note that this wording has changed a lot recently):

> 22 A *constant expression* is either a glvalue core constant expression that refers to an object or a non-immediate function, or a prvalue core constant expression whose value satisfies the following constraints:
>
> (22.1) — each constituent reference refers to an object or a non-immediate function,
>
> (22.2) — no constituent value of scalar type is an indeterminate value ([basic.indet]),
>
> (22.3) — no constituent value of pointer type is a pointer to an immediate function or an invalid pointer value ([basic.compound]), and
>
> (22.4) — no constituent value of pointer-to-member type designates an immediate function.

This code breaks that rule. We have pointers that point to immediate functions, hence we do not have a constant expression, hence we do not have a validly initialized `constexpr` variable.

What do we do now?

§ 4.1 Consteval Variable Escalation

Importantly, `fptrs` is a `static constexpr` variable that is a templated entity, and its initializer — `get_array()` — has consteval-only value. Today, we reject this initialization for the same reason that we rejected the initialization of `ptr` earlier: if that initialization were allowed to succeed, we have regular function pointers, and nothing prevents me from invoking them at runtime. Which would defeat the purpose of the `consteval` specifier.

However.

What if, instead of rejecting the example, the fact that the initializer were a consteval-only value instead led to the escalation of `fptrs` to be `consteval` variable instead of a `constexpr` one? This follows the principle set out in [P2564R3] — there, we had a specialization of a `constexpr` function template that would otherwise be ill-formed, so we make it `consteval`.

We could do the same here! `fptrs` is a `static constexpr` variable in a class template that is initialized to a consteval-only value, so let's escalate it to be a `consteval` variable instead of a `constexpr` one. If we do that, then we have to examine its usage within `visit`, copied here again for convenience:

```
template <class R, class F, class V0, class V1>
constexpr auto visit(F&& f, V0 const& v0, V1 const& v1) -> R {
    using Impl = binary_vtable_impl<R, F, V0, V1>;
```

```
    return Impl::fptrs[v0.index][v1.index]((F&&)f, v0, v1);
}
```

`fptrs` being a `consteval` variable means that the invocation there has to be immediate-escalating. This causes the specialization of `visit` to become a `consteval` function following the same rules as in [P2564R3]. At which point, everything just works.

Put differently — as a `constexpr` variable, `fptrs` was not allowed to be initialized with pointers to immediate functions. But as a `consteval` variable, it can be — since we escalate all invocations of those pointers! Everything just… works, and requires no code changes on the part of the library implementation.

§ 5 Motivating Example: Immediate Functions

Today, we have this behavior:

```
consteval auto always_false() -> bool { return false; }

static_assert(std::not_fn(always_false)());   // OK
static_assert(std::not_fn<always_false>()()); // ill-formed
```

The first `static_assert` itself wasn't always valid, that was corrected by [P2564R3]. But the second `static_assert` is still invalid — perhaps surprisingly so since everything here is evaluated during constant evaluation too.

The issue here is not that `std::not_fn<always_false>()()` evaluates to `false` for some weird reason. Rather, `std::not_fn<always_false>()` itself is ill-formed. Because evaluating that expression involves initializing a constant template parameter with `alway_false`, and that's currently disallowed due to the fact that `always_false` is an immediate function.

This is similar to the failure mode in the variant example, it just that there the `constexpr` variable was explicit (the variable `fptrs`) and here it is implicit (the constant template parameter of `std::not_fn`). And, like the variant example, we are trying to constant-evaluate a call using immediate functions — but it's rejected anyway.

The solution here is similar: if we escalate the template parameter in `std::not_fn<always_false>` to be a `consteval` variable instead of a `constexpr` one, the above code would just work. And, because we do the escalation, we can still catch all the misuses. For instance:

```
consteval auto positive(int i) -> bool { return i > 0; }

void test(int i) {
    // currently: ill-formed, proposed ok
    constexpr auto f = std::not_fn<positive>();

    // this would be ok: the call operator is consteval
```

```
    static_assert(f(-5));

    // this is, importantly, an error
    // otherwise would lead to calling positive at runtime
    bool b = f(i);
```

§ 6 Other Design Questions

There are a few other design questions to discuss.

§ 6.1 Mutability

One question we have to address is, given:

```
consteval int v = 0;
```

What is `decltype(v)`? In Daveed's original proposal in [P0596R1] (Side-effects in constant evaluation: Output and consteval variables), `v` was an `int` that was actually possible to mutate during constant evaluation time. Having compile-time mutable variables would be quite useful to solve some problems, although it is not without its share of complexity — specifically when such mutation is allowed to happen.

While I do think it would be quite valuable to have compile-time mutable variables, I am not pursuing those in this paper for three reasons:

1. They are complicated,
2. I think inherently having a variable declared `consteval` that is mutable is just confusing from a keyword standpoint. It's one thing to have `constinit` — which at least is simply `const`ant `init`ialized. But `consteval` seems a bit strong, and
3. Given that `constexpr` variables can escalate to `consteval` ones, it is important that they don't change types. `constexpr` is `int const`, so `consteval` should be too.

We can always add consteval mutable variables in the future by allowing the declaration:

```
consteval mutable int v = 0;
static_assert(v == 0); // ok
consteval {
  ++v;                 // ok, mutable
}
static_assert(v == 1); // ok, observed mutation
```

Alternatively, because of the potential future of consteval mutable variables, we could enforce that variables declared `consteval` must also be declared `const`. That restriction can be relaxed later. Note that this rule would only be for variables *declared* `consteval`, not those which escalate:

```
consteval int a = 0; // ill-formed
consteval int const b = 0; // ok
```

That is, the two choices are:

1. `consteval`, like `constexpr`, means implicit `const`. `consteval mutable` is currently disallowed, may eventually be allowed.
2. `consteval`, unlike `constexpr`, means `mutable`. `consteval const` would be mandated.

Choosing (2) now doesn't close the door to choosing (1) later, just like you can declare variables `constexpr const` today, it's just redundant. I have a slight preference for (1) and it is what I implemented.

§ 6.2 Rules around `constexpr` Variables

Let's quickly consider:

```
consteval int add(int x, int y) {
    return x + y;
}

constexpr auto a = add;
consteval auto b = add;

constexpr auto c = ^^int;
consteval auto d = ^^int;
```

The status quo is that `a` is ill-formed (as already mentioned) and `c` is proposed okay. `b` and `d` are obviously okay. Is that the right set of rules? There are other alternatives:

— **`c` is ill-formed**. This might be a little surprising to propose, but it actually has merit. If you want a variable that only exists at compile time, declare it `consteval`. Just because we *can* come up with a set of rules that allows `c` (by way of having consteval-only type) but not `a` doesn't mean that we should. Rejecting `c` can also provide a clear error message that it should be `consteval` instead and makes for a simpler set of rules.

— **`a` is well-formed**. We can achieve this by having consteval variable escalation apply for all variables, not just templated ones. But when we were discussing [P2564R3], we didn't do escalation for regular functions then — if you have a function that must be `consteval`, we decided that you should explicitly mark it as such. The same principle should apply here — if `a` has to be `consteval` (and it does), then it should be explicitly labeled as such.

We think the right answer is that only the `consteval` declarations above should be valid. Consider this table:

specifier	usable at run-time	usable at compile-time
(none)	🟢	🔴

specifier	usable at run-time	usable at compile-time
`constexpr`	🟢	🟢
`consteval`	🔴	🟢

This is a very clear way to separate the kinds of specifiers. `consteval` means compile-time-only. `constexpr` means runtime or compile-time. No specifier means runtime-only.

However, that's not the status quo. A more accurate table right now would look more like this:

specifier	usable at run-time	usable at compile-time
(none)	🟢	🔴
`constexpr`	🟡	🟢
`consteval`	🔴	🟡

That is:

— A `constexpr` variable is usable at run-time, except when it's not (when it has consteval-only type. That is: it is a reflection).

— A `consteval` function is usable at compile-time, except when it's not (as we've seen earlier, when the way in which it is used requires initializing a `constexpr` variable).

The second bullet could be done later, but the first bullet requires rejecting `constexpr auto c = ^^int;` in favor of requiring the user to have to write `consteval`. That would be a large (albeit very mechanical) breaking change if we did it later. Doing so would allow us to have the clean model in the earlier table: `constexpr` means usable at runtime or compile-time and `consteval` means usable at compile-time only.

The additional rules we can derive on top of `consteval` variables allow us to more consistently use compile-time only values at compile-time also.

This strikes us as a very valuable place to be in that's superior to the status quo, so we should definitely do it.

§ 6.3 Do we still need consteval-only types?

[P2996R13] introduces the notion of consteval-only type — basically any type that has a `std::meta::info` in it somewhere — to ensure that reflections exist only at compile time. This paper provides an alternative approach to solve the same problem: extend consteval-only to include values of type `std::meta::info`.

This broadly accomplishes the same thing (and would necessitate having `c` be ill-formed in the above example), there are a few cases where the suggested rules would differ though. For example:

```
// variant<info, int> is a "consteval-only type"
// but v does not have "consteval-only value"
constexpr std::variant<std::meta::info, int> v = 42;
```

```
struct C {
    std::meta::info const* p;
};
// C is a "consteval-only type"
// but c does not have "consteval-only value"
auto c = C{.p=nullptr};
```

On the whole, it's definitely important to ensure that reflections do not persist to runtime and do not lead to codegen. These cases don't *actually* have reflections in them though. So perhaps we don't need them the concept of consteval-only type after all. The only other use-case we have is the libc++ sort problem, and it's not even clear that detecting consteval-only types properly solves it.

Basically, consteval-only types was an invention we needed in order to ensure that reflections don't persist to runtime. But consteval-only *values* allows us to do that even better. It also simplifies some other edge-case wording that we have in the standard already. For instance, this example from 7.7 [expr.const]:

```
struct Base { };
struct Derived : Base { std::meta::info r; };

consteval const Base& fn(const Derived& derived) { return derived; }

constexpr Derived obj{.r=^^::};     // OK
constexpr const Derived& d = obj;   // OK
constexpr const Base& b = fn(obj);  // error: not a constant expression
        because Derived
                                    // is a consteval-only type but Base
        is not.
```

We need to reject `b`, despite `Base` not being a consteval-only type, and we need dedicated wording here. But with consteval variables and consteval-only values, rejecting this is very straightforward: `fn(obj)` refers to a consteval variable, so `b` cannot be `constexpr`, that's it. It's not a special case that needs dedicated handling. Which is quite nice!

[P3421R0] (Consteval destructors) is another paper in this space that also seems like what it is really trying to do is come up with a way to produce consteval-only values. Perhaps a consteval destructor would be a way to signal that.

§ 6.4 Consteval-only Allocation

Consider:

```
#include <vector>
consteval std::vector<int> a = {1, 2, 3};
consteval int* p = new int(4);
```

The issue we're trying to solve with non-transient allocation ([P1974R0] (Non-transient constexpr allocation using propconst), [P2670R1] (Non-transient constexpr allocation), and [P3554R0] (Non-transient al-

location with `std::vector` and `std::basic_string`)) relies upon dealing with persistence. How do we persist the constant allocation into runtime in a way that is reliably coherent.

But [P3032R2] (Less transient constexpr allocation) already recognized that there are situations in which a constexpr variable will *not* persist into runtime, so such allocations *could* be allowed. The rule suggested in that paper was `constexpr` variables in immediate function contexts. But `consteval` variables allow for a much clearer, more general approach to the problem: an allocation in an initializer of a `consteval` variable could simply leak — even `p` could be allowed. We would have to adopt the rule suggested in P3032 — that any mutation through the allocation after initialization is disallowed (which we can enforce since the variables live entirely at compile time).

The `consteval` specifier also makes clear that these variables would exist only at compile time, and thus there is no jarring code movement difference that the P3032 rule led to — where you can move a declaration from one context to another and that changes its validity.

Note that this also would help address a usability issue with [P1306R3] (Expansion statements), where we could say that:

```
template for (consteval info r : members_of(type))
```

desugars into declaring the underlying range `consteval`, which seems like a fairly tidy way to resolve that the allocation issue.

Consteval-only allocation can always be adopted later, it is not strictly essential to this proposal, and we're already late.

§ 7 Implementation Experience

I implemented this proposal on top of Dan Katz's p2996 fork of clang, here. Specifically, I implemented:

— variables can be declared `consteval`. This makes them implicitly `const` (see mutability).

— `consteval` variables can be initialized with `consteval`-only values (like reflections and immediate functions). `constexpr` variables *cannot* (see rules.

— templated `constexpr` variables and template parameters whose initializer has consteval-only value become `consteval` variables.

Both the variant and `not_fn` examples compile as-is with no code changes.

§ 8 Proposal

This paper proposes:

1. introducing the notion of consteval-only value,
2. removing the notion of consteval-only type,

3. introducing consteval variables (which are implicitly `const`),
4. allowing certain constexpr variables (those with consteval-only value) to escalate to consteval variables.

Currently, the only kinds of consteval-only value is a pointer (or reference) to immediate function and consteval-only types (i.e. reflections). This paper directly also adds consteval variables.

§ 8.1 Wording

[Editor's note: We should endeavor to change all of our "immediate" terms to just be "consteval" terms. Consteval function, consteval invocation, etc. But for now, we're sticking with "immediate"].

Remove consteval-only type from 6.8.1 [basic.types.general]/12:

12 ~~A type is *consteval-only* if it is either `std::meta::info` or a type compounded from a consteval-only type ([basic.compound]). Every object of consteval-only type shall be~~

(12.1) — ~~the object associated with a constexpr variable or a subobject thereof,~~

(12.2) — ~~a template parameter object ([temp.param]) or a subobject thereof, or~~

(12.3) — ~~an object whose lifetime begins and ends during the evaluation of a core constant expression.~~

Change 7.7 [expr.const]

6 A variable `v` is *constant-initializable* if

(6.1) — the full-expression of its initialization is ~~a~~ an immediate constant expression when interpreted as a *constant-expression* and is a constant expression if `v` is not an immediate variable, [*Note 2:* Within this evaluation, `std::is_constant_evaluated()` ([meta.const.eval]) returns `true`. — *end note*] and

(6.2) — immediately after the initializing declaration of `v`, the object or reference `x` declared by `v` is constexpr-representable, and

(6.3) — if `x` has static or thread storage duration, `x` is constexpr-representable at the nearest point whose immediate scope is a namespace scope that follows the initializing declaration of `v`.

7 A constant-initializable variable is *constant-initialized* if either it has an initializer or its type is const-default-constructible ([dcl.init.general]).

8 A variable is *potentially-constant* if

(8.1) — it is constexpr ,

(8.2) — it is consteval, or

(8.3) — it has reference or non-volatile const-qualified integral or enumeration type.

[Drafting note: The above is now made a bulleted list, with the addition of the third condition.]

9 A constant-initialized potentially-constant variable *V* is usable in constant expressions at a point *P* if *V*'s initializing declaration *D* is reachable from *P* and

(9.1) — *V* is constexpr or consteval,

(9.2) — *V* is not initialized to a TU-local value, or

(9.3) — *P* is in the same translation unit as *D*.

[…]

w An *immediate value* is a value that satisfies any of the following:

(w.1) — it is a reflection,

(w.2) — it is an immediate function,

(w.3) — any constituent reference refers to an immediate function or an immediate object,

(w.4) — any constituent pointer points to an immediate function or an immediate object,

(w.5) — any constituent value is a reflection, or

(w.6) — any constituent value of pointer-to-member type designates an immediate function.

x An *immediate object* is an object that was either initialized by an immediate value or declared by an immediate variable.

y An *immediate variable* is

(y.1) — a variable declared with the `consteval` specifier, or

(y.2) — a variable that results from the instantiation of a templated entity declared with the `constexpr` specifier whose initialization is an immediate constant expression that is not a constant expression (see below).

z An *immediate constant expression* is either a glvalue core constant expression that refers to an object or a function, or a prvalue core constant expression whose value satisfies the following constraints:

(z.1) — each constituent reference refers to an object or a function,

(z.2) — no constituent value of scalar type is an indeterminate or erroneous value ([basic.indet]), and

(z.3) — no constituent value of pointer type has an invalid pointer value ([basic.compound]).

22 A *constant expression* is either

(22.1) — a glvalue immediate ~~core~~ constant expression *E* ~~for which~~ that refers to a non-immediate object or non-immediate function, or

(22.1.1) — ~~*E* refers to a non-immediate function,~~

(22.1.2) — ~~*E* designates an object *o*, and if the complete object of *o* is of consteval-only type then so is *E*,~~

~~[*Example 1:*~~

```
struct Base { };
struct Derived : Base { std::meta::info r; };

consteval const Base& fn(const Derived& derived) { return derived; }

constexpr Derived obj{.r=^^::}; // OK
constexpr const Derived& d = obj; // OK
constexpr const Base& b = fn(obj); // error: not a constant expression
  // because Derived is a consteval-only type but Base is not.
```

~~— *end example*]~~

(22.2) — a prvalue ~~core~~ immediate constant expression whose result value object ([basic.lval]) ~~satisfies the following constraints~~ does not have immediate value.

(22.2.1) — ~~each constituent reference refers to an object or a non-immediate function,~~

(22.2.2) — ~~no constituent value of scalar type is an indeterminate or erroneous value ([basic.indet]),~~

(22.2.3) — ~~no constituent value of pointer type is a pointer to an immediate function or an invalid pointer value ([basic.compound]), and~~

(22.2.4) — ~~no constituent value of pointer-to-member type designates an immediate function.~~

(22.2.5) — ~~unless the value is of consteval-only type,~~

(22.2.5.1) — ~~no constituent value of pointer-to-member type points to a direct member of a consteval-only class type~~

(22.2.5.2) — ~~no constituent value of pointer type points to or past an object whose complete object is of consteval-only type, and~~

(22.2.5.3) — ~~no constituent reference refers to an object whose complete object is of consteval-only type.~~

[…]

24 An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

(24.1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function,

(24.2) — it is a subexpression of a manifestly constant-evaluated expression or conversion, or

(24.3) — its enclosing statement is enclosed ([stmt.pre]) by the *compound-statement* of a consteval if statement ([stmt.if]).

An ~~invocation~~ expression is an ~~*immediate invocation*~~ *immediate evaluation* if it ~~is a potentially-evaluated explicit or implicit invocation of an immediate function and~~ is not in an immediate function context and either

(24.4) — it is a potentially-evaluated explicit or implicit invocation of an immediate function or

(24.5) — it is an lvalue-to-rvalue conversion applied to an immediate variable.

An aggregate initialization is an immediate ~~invocation~~ evaluation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression.

25 A potentially-evaluated expression or conversion is *immediate-escalating* if it is neither initially in an immediate function context nor a subexpression of an immediate ~~invocation~~ evaluation, and

(25.1) — it is an *id-expression* or *splice-expression* that designates an immediate function or immediate variable,

(25.2) — it is an immediate ~~invocation~~ evaluation that is not a constant expression, or

(25.3) — ~~it is of consteval-only type ([basic.types.general])~~ it has an immediate value.

26 An *immediate-escalating* function is […]

27 An *immediate function* is […]

Change 9.2.6 [dcl.constexpr] to account for `consteval` variables:

1 The `constexpr` and `consteval` specifiers shall be applied only to the definition of a variable or variable template, a structured binding declaration, or the declaration of a function or function template. ~~The `consteval` specifier shall be applied only to the declaration of a function or function template.~~ A function or static data member declared with the `constexpr` or `consteval` specifier on its first declaration is implicitly an inline function or variable ([dcl.inline]). If any declaration of a function or function template has a `constexpr` or `consteval` specifier, then all its declarations shall contain the same specifier.

[…]

6 A `constexpr` or `consteval` specifier used in an object declaration declares the object as `const`. Such an object shall have literal type and shall be initialized. A `constexpr` or `consteval` variable shall be constant-initializable ([expr.const]). A `constexpr` or `consteval`

variable that is an object, as well as any temporary to which a constexpr reference is bound, shall have constant destruction.

Change 13.4.3 [temp.arg.nontype] to refer back to [expr.const]:

1 A template argument *E* for a constant template parameter with declared type *T* shall be such that the invented declaration

```
T x = E ;
```

satisfies the semantic constraints for the definition of a constexpr variable with static storage duration ([dcl.constexpr]). If T contains a placeholder type ([dcl.spec.auto]) or a placeholder for a deduced class type ([dcl.type.class.deduct]), the type of the parameter is deduced from the above declaration. [Note 1: E is a template-argument or (for a default template argument) an initializer-clause. — end note] If the parameter type thus deduced is not permitted for a constant template parameter ([temp.param]), the program is ill-formed.

2 The value of a constant template parameter P of (possibly deduced) type T is determined from its template argument A as follows. If T is not a class type and A is not a braced-init-list, A shall be a converted constant expression ([expr.const]) of type T; the value of P is A (as converted).

3 Otherwise, a temporary variable

```
constexpr T v = A;
```

is introduced. The lifetime of v ends immediately after initializing it and any template parameter object (see below). For each such variable, the id-expression v is termed a candidate initializer.

* [*Note 3:* This temporary variable may be an immediate variable if the initialization is an immediate constant expression that is not a constant expression ([expr.const]) — *end note*]

§ 8.2 Feature-test Macro

Bump `__cpp_consteval` in 15.12 [cpp.predefined]:

```
- __cpp_consteval 202406L
+ __cpp_consteval 20XXXXL
```

§ 9 Acknowledgments

An earlier draft revision of the paper proposed something much narrower — simply allowing pointers to immediate functions to persist, if those exists as part of `static constexpr` variables in immediate functions. Richard Smith suggested that we generalize this further. That suggestion led us to the much better design that this paper now proposes. Thank you, Richard.

§ 10 References

[LWG4197] Jiang An. 2025-01-26. Complexity of `std::visit` with immediate functions.
https://wg21.link/lwg4197

[P0596R1] Daveed Vandevoorde. 2019-10-08. Side-effects in constant evaluation: Output and consteval variables.
https://wg21.link/p0596r1

[P1073R3] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018-11-06. Immediate functions.
https://wg21.link/p1073r3

[P1306R3] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2024-10-14. Expansion statements.
https://wg21.link/p1306r3

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.
https://wg21.link/p1974r0

[P2564R3] Barry Revzin. 2022-11-11. consteval needs to propagate up.
https://wg21.link/p2564r3

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.
https://wg21.link/p2670r1

[P2996R13] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-06-18. Reflection for C++26.
https://wg21.link/p2996r13

[P3032R2] Barry Revzin. 2024-04-16. Less transient constexpr allocation.
https://wg21.link/p3032r2

[P3421R0] Ben Craig. 2024-10-12. Consteval destructors.
https://wg21.link/p3421r0

[P3496R0] Barry Revzin. 2025-01-06. Immediate-Escalating Expressions.
https://wg21.link/p3496r0

[P3554R0] Peter Dimov and Barry Revzin. 2025-01-05. Non-transient allocation with `std::vector` and `std::basic_string`.
https://wg21.link/p3554r0

[P3603R0] Barry Revzin. 2025-03-13. Consteval-only Values and Consteval Variables.
https://wg21.link/p3603r0